

GSAP (GreenSock Animation Platform) Documentation

Table of Contents

1. [Introduction](#)
 2. [Installation & Setup](#)
 3. [Core Concepts](#)
 4. [Basic Methods](#)
 5. [Timeline](#)
 6. [Easing Functions](#)
 7. [Advanced Features](#)
 8. [Plugins](#)
 9. [Performance Best Practices](#)
 10. [Common Use Cases](#)
 11. [Troubleshooting](#)
-

Introduction

GSAP (GreenSock Animation Platform) is a robust JavaScript animation library that enables developers to create high-performance animations for the web. It's widely regarded as the gold standard for web animation due to its:

- **High Performance:** Up to 20x faster than jQuery
- **Cross-browser Compatibility:** Works in all browsers
- **Ease of Use:** Intuitive API and extensive documentation
- **Flexibility:** Animates CSS properties, SVG, canvas, and more
- **Professional Grade:** Used by major companies worldwide

Key Features

- Animate any numeric property of any JavaScript object
 - Robust timeline system for complex sequences
 - Advanced easing options
 - Morphing capabilities for SVG paths
 - 3D transforms and hardware acceleration
 - Responsive animation controls
-

Installation & Setup

CDN Installation

html

```
<!-- Core GSAP library -->  
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.2/gsap.min.js"> </script>  
  
<!-- Additional plugins (optional) -->  
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.2/ScrollTrigger.min.js"> </script>  
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.2/TextPlugin.min.js"> </script>
```

NPM Installation

bash

```
npm install gsap
```

javascript

```
// ES6 Import  
import { gsap } from "gsap";  
import { ScrollTrigger } from "gsap/ScrollTrigger";  
  
// Register plugins  
gsap.registerPlugin(ScrollTrigger);
```

Basic Setup

html

```
<!DOCTYPE html>
<html>
<head>
  <title>GSAP Example</title>
  <style>
    .box {
      width: 100px;
      height: 100px;
      background: #ff6b6b;
      margin: 50px;
    }
  </style>
</head>
<body>
  <div class="box"></div>

  <script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.2/gsap.min.js"></script>
  <script>
    gsap.to(".box", {duration: 2, x: 100, rotation: 360});
  </script>
</body>
</html>
```

Core Concepts

Animation Methods

GSAP provides four main methods for creating animations:

1. **gsap.to()** - Animates FROM current state TO specified values
2. **gsap.from()** - Animates FROM specified values TO current state
3. **gsap.fromTo()** - Animates FROM specified values TO other specified values
4. **gsap.set()** - Immediately sets properties without animation

Target Selection

GSAP accepts various target types:

- CSS selectors: `".class"`, `"#id"`, `"div"`
- DOM elements: `document.getElementById("myId")`
- Arrays: `[element1, element2, element3]`
- Objects: Any JavaScript object with numeric properties

Properties

You can animate virtually any numeric property:

- **CSS Properties:** x, y, scale, rotation, opacity, width, height
 - **Transform Properties:** scaleX, scaleY, rotationX, rotationY, skewX, skewY
 - **CSS3 Properties:** borderRadius, boxShadow, textShadow
 - **Custom Properties:** Any numeric property of any object
-

Basic Methods

gsap.to()

Animates elements from their current state to new values.

```
javascript

// Basic usage
gsap.to(".box", {duration: 2, x: 100, y: 50});

// Multiple properties
gsap.to(".box", {
  duration: 3,
  x: 200,
  y: 100,
  scale: 1.5,
  rotation: 360,
  opacity: 0.5,
  backgroundColor: "#00ff00"
});

// Multiple targets
gsap.to([".box1", ".box2", ".box3"], {
  duration: 1,
  y: -100,
  stagger: 0.2 // Delay between each element
});
```

gsap.from()

Animates elements from specified values to their current state.

```
javascript
```

```
// Element starts at x: -100 and animates to its current position
```

```
gsap.from(".box", {duration: 2, x: -100, opacity: 0});
```

```
// Useful for entrance animations
```

```
gsap.from(".title", {  
  duration: 1,  
  y: -50,  
  opacity: 0,  
  ease: "bounce.out"  
});
```

gsap.fromTo()

Defines both starting and ending values.

```
javascript
```

```
gsap.fromTo(".box",  
  {x: -100, opacity: 0}, // FROM values  
  {duration: 2, x: 100, opacity: 1} // TO values  
);
```

```
// More complex example
```

```
gsap.fromTo(".card",  
  {  
    scale: 0,  
    rotation: -180,  
    opacity: 0  
  },  
  {  
    duration: 1.5,  
    scale: 1,  
    rotation: 0,  
    opacity: 1,  
    ease: "back.out(1.7)"  
  }  
);
```

gsap.set()

Immediately sets properties without animation.

```
javascript
```

```
// Set initial state
gsap.set(".box", {x: 100, y: 50, scale: 0.5});

// Set multiple elements
gsap.set([".item1", ".item2"], {opacity: 0, y: 30});
```

Timeline

Timelines allow you to create complex sequences and have precise control over timing.

Basic Timeline

```
javascript

// Create a timeline
let tl = gsap.timeline();

// Add animations to the timeline
tl.to(".box1", {duration: 1, x: 100})
.to(".box2", {duration: 1, y: 100}) // Starts after previous ends
.to(".box3", {duration: 1, rotation: 360}); // Starts after previous ends
```

Timeline with Position Parameter

```
javascript

let tl = gsap.timeline();

tl.to(".box1", {duration: 1, x: 100})
.to(".box2", {duration: 1, y: 100, "-=0.5"}) // Start 0.5s before previous ends
.to(".box3", {duration: 1, rotation: 360, "+=0.5"}) // Start 0.5s after previous ends
.to(".box4", {duration: 1, scale: 2, 0.5}) // Start at 0.5s from timeline start
.to(".box5", {duration: 1, opacity: 0, "<"}) // Start with previous animation
.to(".box6", {duration: 1, skewX: 45, ">"}); // Start after previous ends
```

Timeline Labels

```
javascript
```

```
let tl = gsap.timeline();

tl.to(".box1", {duration: 1, x: 100})
  .addLabel("midpoint")
  .to(".box2", {duration: 1, y: 100})
  .to(".box3", {duration: 1, rotation: 360, "midpoint"}) // Start at label
  .addLabel("end");
```

Timeline Control

```
javascript

let tl = gsap.timeline({paused: true}); // Start paused

// Control methods
tl.play();    // Play timeline
tl.pause();   // Pause timeline
tl.reverse(); // Reverse timeline
tl.restart(); // Restart from beginning
tl.seek(1.5); // Jump to 1.5 seconds
tl.progress(0.5); // Jump to 50% progress
tl.timeScale(2); // Double the speed
```

Timeline Callbacks

```
javascript

let tl = gsap.timeline({
  onComplete: function() {
    console.log("Timeline completed");
  },
  onUpdate: function() {
    console.log("Progress: " + this.progress());
  },
  onRepeat: function() {
    console.log("Timeline repeated");
  },
  repeat: 2,
  repeatDelay: 1
});
```

Easing Functions

Easing controls the rate of change during animations, making them feel more natural and engaging.

Built-in Easing Types

Linear

javascript

```
gsap.to(".box", {duration: 2, x: 100, ease: "none"}); // Linear
```

Power

javascript

```
gsap.to(".box", {duration: 2, x: 100, ease: "power2.out"});  
gsap.to(".box", {duration: 2, x: 100, ease: "power3.in"});  
gsap.to(".box", {duration: 2, x: 100, ease: "power4.inOut"});
```

Back

javascript

```
gsap.to(".box", {duration: 2, x: 100, ease: "back.out(1.7)"});  
gsap.to(".box", {duration: 2, x: 100, ease: "back.in(3)"});  
gsap.to(".box", {duration: 2, x: 100, ease: "back.inOut(1.7)"});
```

Elastic

javascript

```
gsap.to(".box", {duration: 2, x: 100, ease: "elastic.out(1, 0.3)"});  
gsap.to(".box", {duration: 2, x: 100, ease: "elastic.in(1, 0.5)"});  
gsap.to(".box", {duration: 2, x: 100, ease: "elastic.inOut(1, 0.3)"});
```

Bounce

javascript

```
gsap.to(".box", {duration: 2, x: 100, ease: "bounce.out"});  
gsap.to(".box", {duration: 2, x: 100, ease: "bounce.in"});  
gsap.to(".box", {duration: 2, x: 100, ease: "bounce.inOut"});
```

Circ, Expo, Sine

javascript


```
gsap.to(".box", {duration: 2, x: 100, ease: "circ.out"});  
gsap.to(".box", {duration: 2, x: 100, ease: "expo.inOut"});  
gsap.to(".box", {duration: 2, x: 100, ease: "sine.in"});
```

Custom Cubic Bezier

```
javascript  
  
gsap.to(".box", {  
  duration: 2,  
  x: 100,  
  ease: "cubic-bezier(0.25, 0.46, 0.45, 0.94)"  
});
```

Steps

```
javascript  
  
gsap.to(".box", {duration: 2, x: 100, ease: "steps(10)"});
```

Advanced Features

Stagger

Animate multiple elements with delays between each.

```
javascript
```

```
// Basic stagger
gsap.to(".item", {
  duration: 1,
  y: -100,
  stagger: 0.1 // 0.1s delay between each
});

// Advanced stagger
gsap.to(".item", {
  duration: 1,
  y: -100,
  stagger: {
    each: 0.1,
    from: "center", // "start", "end", "edges", "random", "center"
    grid: [7, 15], // Stagger in a grid pattern
    axis: "y", // "x" or "y"
    ease: "power2.inOut",
    repeat: -1,
    yoyo: true
  }
});
```

Repeat and Yoyo

```
javascript

gsap.to(".box", {
  duration: 1,
  x: 100,
  repeat: 3, // Repeat 3 times (4 total plays)
  yoyo: true, // Reverse on alternate repeats
  repeatDelay: 0.5 // Wait 0.5s between repeats
});

// Infinite repeat
gsap.to(".box", {
  duration: 2,
  rotation: 360,
  repeat: -1, // Infinite
  ease: "none"
});
```

Callbacks

```
javascript
```

```
gsap.to(".box", {
  duration: 2,
  x: 100,
  onStart: function() {
    console.log("Animation started");
  },
  onUpdate: function() {
    console.log("Progress: " + this.progress());
  },
  onComplete: function() {
    console.log("Animation completed");
    // Chain another animation
    gsap.to(this.targets(), {duration: 1, y: 100});
  },
  onReverseComplete: function() {
    console.log("Reverse completed");
  }
});
```

Delays and Random

```
javascript

// Fixed delay
gsap.to(".box", {duration: 1, x: 100, delay: 0.5});

// Random delay
gsap.to(".box", {
  duration: 1,
  x: 100,
  delay: "random(0, 2)" // Random delay between 0 and 2 seconds
});

// Random values
gsap.to(".particle", {
  duration: "random(1, 3)",
  x: "random(-200, 200)",
  y: "random(-200, 200)",
  scale: "random(0.5, 2)"
});
```

Plugins

GSAP offers numerous plugins to extend functionality. Some are free, others require a paid license.

Free Plugins

ScrollTrigger

```
javascript

gsap.registerPlugin(ScrollTrigger);

gsap.to(".box", {
  x: 400,
  duration: 3,
  scrollTrigger: {
    trigger: ".box",
    start: "top 80%",
    end: "bottom 20%",
    scrub: true, // Tie animation to scroll position
    markers: true // Show trigger markers (for debugging)
  }
});
```

TextPlugin

```
javascript

gsap.registerPlugin(TextPlugin);

gsap.to(".text", {
  duration: 2,
  text: "This text will animate in character by character!"
});
```

MotionPathPlugin

```
javascript

gsap.registerPlugin(MotionPathPlugin);

gsap.to(".box", {
  duration: 5,
  motionPath: {
    path: "#path", // SVG path element
    autoRotate: true,
    alignOrigin: [0.5, 0.5]
  }
});
```

Premium Plugins (Require License)

MorphSVGPlugin

```
javascript

gsap.to("#star", {
  duration: 2,
  morphSVG: "#circle"
});
```

DrawSVGPlugin

```
javascript

gsap.fromTo(".line",
  {drawSVG: "0%"},
  {duration: 2, drawSVG: "100%"}
);
```

SplitText

```
javascript

let split = new SplitText(".text", {type: "chars"});
gsap.from(split.chars, {
  duration: 0.8,
  opacity: 0,
  scale: 0,
  y: 80,
  rotationX: 180,
  transformOrigin: "0% 50% -50",
  ease: "back.out",
  stagger: 0.01
});
```

Performance Best Practices

Use Transform Properties

```
javascript
```

```
//  Good - Uses transforms (GPU accelerated)
```

```
gsap.to(".box", {duration: 1, x: 100, y: 50, scale: 1.5});
```

```
//  Avoid - Triggers layout/paint
```

```
gsap.to(".box", {duration: 1, left: "100px", top: "50px", width: "150px"});
```

Batch Similar Operations

```
javascript
```

```
//  Good - Single animation for multiple elements
```

```
gsap.to(".item", {duration: 1, x: 100, stagger: 0.1});
```

```
//  Avoid - Multiple individual animations
```

```
document.querySelectorAll(".item").forEach((item, i) => {  
  gsap.to(item, {duration: 1, x: 100, delay: i * 0.1});  
});
```

Use will-change CSS Property

```
css
```

```
.animated-element {  
  will-change: transform, opacity;  
}
```

Optimize with force3D

```
javascript
```

```
gsap.to(".box", {  
  duration: 1,  
  x: 100,  
  force3D: true // Force hardware acceleration  
});
```

Kill Unused Animations

```
javascript
```

```
let animation = gsap.to(".box", {duration: 5, x: 100});
```

// Kill if no longer needed

```
animation.kill();
```

// Kill all animations on an element

```
gsap.killTweensOf(".box");
```

Common Use Cases

Loading Animation

javascript

```
let tl = gsap.timeline();
```

```
tl.from(".logo", {duration: 1, scale: 0, ease: "back.out(1.7)"})
```

```
.from(".loading-text", {duration: 0.5, opacity: 0, y: 20, "-=0.5"})
```

```
.to(".loader", {duration: 0.5, opacity: 0, delay: 1})
```

```
.set(".loader", {display: "none"});
```

Card Hover Effect

javascript

```
function setupCardHover() {  
  const cards = document.querySelectorAll(".card");  
  
  cards.forEach(card => {  
    card.addEventListener("mouseenter", () => {  
      gsap.to(card, {  
        duration: 0.3,  
        y: -10,  
        scale: 1.05,  
        boxShadow: "0 20px 40px rgba(0,0,0,0.2)",  
        ease: "power2.out"  
      });  
    });  
  
    card.addEventListener("mouseleave", () => {  
      gsap.to(card, {  
        duration: 0.3,  
        y: 0,  
        scale: 1,  
        boxShadow: "0 5px 15px rgba(0,0,0,0.1)",  
        ease: "power2.out"  
      });  
    });  
  });  
}
```

Scroll-Triggered Animations

javascript


```
gsap.registerPlugin(ScrollTrigger);

// Reveal elements as they scroll into view
gsap.utils.toArray(".reveal").forEach((element, i) => {
  gsap.fromTo(element,
    {
      opacity: 0,
      y: 100
    },
    {
      duration: 1,
      opacity: 1,
      y: 0,
      ease: "power2.out",
      scrollTrigger: {
        trigger: element,
        start: "top 85%",
        end: "bottom 15%",
        toggleActions: "play none none reverse"
      }
    }
  );
});
```

Page Transition

javascript

```
function pageTransition() {  
  const tl = gsap.timeline();  
  
  tl.to(".transition-overlay", {  
    duration: 0.5,  
    scaleY: 1,  
    transformOrigin: "top",  
    ease: "power2.inOut"  
  })  
  .call(() => {  
    // Change page content here  
    window.location.href = newURL;  
  })  
  .to(".transition-overlay", {  
    duration: 0.5,  
    scaleY: 0,  
    transformOrigin: "bottom",  
    ease: "power2.inOut"  
  });  
  
  return tl;  
}
```

Infinite Animation Loop

javascript

```
function createFloatingAnimation() {  
  gsap.to(".floating-element", {  
    y: -20,  
    duration: 2,  
    ease: "power1.inOut",  
    yoyo: true,  
    repeat: -1  
  });  
  
  gsap.to(".floating-element", {  
    x: 10,  
    duration: 3,  
    ease: "power1.inOut",  
    yoyo: true,  
    repeat: -1  
  });  
  
  gsap.to(".floating-element", {  
    rotation: 5,  
    duration: 4,  
    ease: "power1.inOut",  
    yoyo: true,  
    repeat: -1  
  });  
}
```

Troubleshooting

Common Issues and Solutions

Animation Not Working

javascript

//  Problem: Element not found

```
gsap.to(".non-existent", {duration: 1, x: 100});
```

//  Solution: Check if element exists

```
const element = document.querySelector(".box");
```

```
if (element) {
```

```
  gsap.to(element, {duration: 1, x: 100});
```

```
}
```

Performance Issues

javascript

```
// ❌ Problem: Animating expensive properties
gsap.to(".box", {duration: 1, width: "200px", height: "200px"});

// ✅ Solution: Use transforms
gsap.to(".box", {duration: 1, scaleX: 2, scaleY: 2});
```

Conflicting Animations

javascript

```
// ❌ Problem: Multiple animations on same property
gsap.to(".box", {duration: 2, x: 100});
gsap.to(".box", {duration: 1, x: 200}); // Conflicts with above

// ✅ Solution: Kill previous animations or use timeline
gsap.killTweensOf(".box");
gsap.to(".box", {duration: 1, x: 200});
```

CSS Conflicts

css

```
/* ❌ Problem: CSS transitions interfere */
.box {
  transition: all 0.3s ease;
}

/* ✅ Solution: Remove CSS transitions on animated elements */
.box {
  transition: none;
}
```

Debugging Tips

Enable Markers for ScrollTrigger

javascript

```
ScrollTrigger.config({
  markers: true
});
```

Log Animation Progress

```
javascript
```

```
gsap.to(".box", {  
  duration: 2,  
  x: 100,  
  onUpdate: function() {  
    console.log("Progress:", this.progress());  
    console.log("Current x:", gsap.getProperty(".box", "x"));  
  }  
});
```

Check GSAP Version

```
javascript
```

```
console.log("GSAP Version:", gsap.version);
```

Browser Compatibility

Autoprefixer for CSS Properties

GSAP automatically handles vendor prefixes for CSS properties, but ensure your CSS build process includes autoprefixer for static styles.

IE11 Support

```
javascript
```

```
// For IE11 compatibility, you may need polyfills  
// Include these before GSAP:  
// - Promise polyfill  
// - Array.from polyfill  
// - Object.assign polyfill
```

Conclusion

GSAP is an incredibly powerful and flexible animation library that can handle everything from simple UI transitions to complex interactive experiences. Key takeaways:

1. **Start Simple:** Begin with basic `gsap.to()`, `gsap.from()`, and `gsap.set()` methods
2. **Use Timelines:** For complex sequences, timelines provide better control and organization
3. **Optimize Performance:** Prefer transform properties and batch operations when possible
4. **Explore Plugins:** Extend functionality with ScrollTrigger, TextPlugin, and premium plugins

5. **Practice:** The more you use GSAP, the more intuitive it becomes

Resources

- Official Documentation: <https://greensock.com/docs/>
- Learning Platform: <https://greensock.com/learning/>
- Community Forum: <https://greensock.com/forums/>
- CodePen Examples: Search "GSAP" on CodePen for inspiration

Version Information

This documentation is based on GSAP 3.12.2. Always refer to the official documentation for the most up-to-date information and new features.

Happy animating with GSAP! 🚀